



ICARUS FUND, LLC

Internal Document - Internship (Quant&Data Track)

Momentum Backtesting Task

Confidential – NDA Protected

This document is confidential and proprietary to Icarus Fund, LLC. Unauthorized distribution, disclosure, or reproduction is strictly prohibited under the Non-Disclosure Agreement signed upon internship onboarding.

Introduction

The objective of this task is to back-test an option momentum strategy across equity names with listed options. This involves building delta-neutral straddles, tracking historical returns, forming long-short portfolios based on momentum signals, and evaluating performance metrics over time.

As of now, we have uploaded option and stock data covering the **top 20 most liquid names** based on option trading activity. The dataset includes:

- **Top 1–10 Underlyings:** AAPL [Apple], TSLA [Tesla], AMZN [Amazon], NVDA [NVIDIA], MSFT [Microsoft], META [Meta Platforms], AMD [Advanced Micro Devices], GOOGL [Alphabet], NFLX [Netflix], BABA [Alibaba].
- **Top 11–20 Underlyings:** PLTR [Palantir], AVGO [Broadcom], MSTR [MicroStrategy], SOFI [SoFi Technologies], MARA [Marathon Digital], INTC [Intel], COIN [Coinbase], NIO [NIO Inc.], F [Ford], RIVN [Rivian].

These names are sufficient to build and validate your initial code base across multiple sectors and volatility profiles.

Note: The long-term goal is to extend this strategy to the full universe of optionable tickers in the OptionMetrics dataset. Once your implementation is robust and tested on the top 20 underlyings, we will transition to running the back-test across all available option roots.

Tips: Reading the Parquet File in Python: To read the Parquet file into Python using `pandas`, use the following code snippet:

```
import pandas as pd

# Load stock-side dataset from Parquet
stock_data = pd.read_parquet("path_to_your_file/stock_data.parquet")

# Preview
print(stock_data.head())
```

Implementation Tip: Please keep your code modular and scalable to accommodate this broader application later on. For example, design your data loading and filtering functions to work generically across all tickers and dates.

Questions and Support

If anything is unclear or you would like to ask for clarification, please fill out the question form using the link below. We monitor submissions regularly and will get back to you as soon as possible.

Question Form: [Click Here](#).

Once you have completed your work for the week, please submit your final PowerPoint deck using the official submission link below. Be sure to double-check formatting, naming conventions, and completeness before submission.

Submission Link: [Click Here](#).

Confidentiality

As with all Icarus Fund assignments, this task and its materials are under NDA. Do not share dataset contents or any intermediate results externally.

Datasets

The dataset is accessible via Dropbox:

[Option Data Download Here,](#)

[Stock Data Download Here,](#)

[Option & Stock Data Link Table Download Here,](#)

1. Option-Side Dataset: (2018–2022, Five Year Data in ZIP file, since the file size is large, so please make sure that you download once at your computer and load it to Python so you don't have to download again):

- Includes end-of-day bid-ask quotes for call and put options.
- Used to:
 - Identify candidate options for delta-neutral straddles.
 - Compute option returns and construct momentum portfolios.
 - Filter contracts based on delta, bid-ask spread, and open interest.

2. Stock-Side Dataset(in csv or parquet file):

- Contains:
 - Daily stock prices (adjusted for splits and dividends)
 - Daily returns
 - Trading volume
 - Manually calculated market capitalization
 - Split adjustment factors
- Used to:
 - Verify option return calculations.
 - Compute historical volatility and normalization metrics.
 - Support any dynamic hedging or firm-level analysis.

Variable Description Table

Option-Side Variables from Option_Price File

Variable	Description
security_id	Integer identifier for the underlying security
date	Date of the quote (trading day)
symbol	Option symbol (root + series)
symbol_flag	1 = OSI compliant; 0 = pre-OSI symbol format
strike_price	Strike price multiplied by 1000
ex_date	Expiration date of the option
cp_flag	Option type: C for call, P for put
best_bid	Highest bid across exchanges (EOD or snapshot quote)
best_offer	Lowest ask across exchanges (EOD or snapshot quote)
last_trade_date	Date the option last traded
volume	Total trading volume (number of contracts)
open_interest	Total number of outstanding contracts (lagged 1 day post-2000)
special_settlement	0 = standard; 1 = fractional or cash component; E = erroneous expiration
implied_volatility	Black-Scholes implied volatility (not for non-standard contracts)
delta	Change in option price for a \$1 change in underlying
gamma	Change in delta for a \$1 change in underlying
vega	Change in option price (in cents) for a 1% change in volatility
theta	Change in option price as time passes (in dollars per year)
option_id	Unique identifier for the option contract
adjustment_factor	Product of all adjustments due to splits/dividends as of date
am_settlement	0 = expires at market close; 1 = expires at market open
contract_size	Number of underlying shares delivered (usually 100)
expiry_indicator	blank = standard, w = weekly, d = daily, m = EOM

Stock-Side Variables

Variable	Description
permno	Unique permanent identifier for the stock
date	Trading date
ret	Raw daily return
prc	Unadjusted stock price
vol	Trading volume
shrout	Shares outstanding (in thousands)
cfacpr	Price adjustment factor (e.g., for splits)
cfacshr	Share adjustment factor
ticker	Ticker symbol
adj_prc	Price adjusted for splits and dividends
adj_ret	Return adjusted for splits and dividends
adj_vol	Adjusted volume (if adjusted for splits)
adj_shrout	Adjusted shares outstanding
mktcap	Market capitalization (price × shares)

Option-Side Variables Example

Variable	Value	Description
secid	101594	OptionMetrics security ID (links to underlying stock, here AAPL)
date	2023-08-31	Observation date for the option data
symbol	AAPL 230901C100000	Option symbol (underlying: AAPL, expiration: 2023-09-01, call at \$100.00)
symbol_flag	1	Indicator of primary option listing (1 = standard contract)
exdate	2023-09-01	Expiration date of the option contract

last_date	2023-08-24	Last trade date recorded before expiration
cp_flag	C	Option type: 'C' for call, 'P' for put
strike_price	100000	Strike price is \$100.00 (divided by 1000)
best_bid	87.4	Best bid quote (highest price someone is willing to pay for the option)
best_offer	88.9	Best ask/offer quote (lowest price someone is willing to sell the option)
volume	0	Number of contracts traded on that day (zero volume implies no trades)
open_interest	61	Number of outstanding contracts (not yet exercised or expired)
impl_volatility	5.745755	Black-Scholes implied volatility (annualized, very high in this case)
delta	0.987722	Sensitivity of option price to underlying asset price (\$1 increase in stock increases call by \$0.9877)
gamma	0.000565	Sensitivity of delta to changes in stock price (second derivative)
vega	0.314162	Sensitivity of option price to a 1% increase in implied volatility
theta	-333.9258	Daily decay in option value due to time passing (negative)
optionid	155643166	Unique identifier for the specific option contract

Linking Option Data and Stock Data

To connect the option-side data (identified by `secid`) and the stock-side data (identified by `permno`), we provide a mapping file:

File Name: **us_stock_permno_with_secid.csv**

This file contains a mapping for each equity ticker in the dataset, enabling you to join daily straddle returns from OptionMetrics with stock-level information (prices, returns, volume, shares outstanding, etc.) available in the Parquet file.

How to Use:

- Use this file to map the `secid` from the option dataset to the `permno` in the stock dataset.
- This allows accurate merges across asset types using common identifiers and consistent time alignment.

Variable Descriptions:

Best Practice: Always check that the date of the option record falls within the `sdate` and `edate` window to ensure the ticker-`permno`-`secid` link is valid at that time.

Table: Variable Description in `us_stock_permno_with_secid.csv`

Variable	Description
<code>permno</code>	Permanent company identifier used in CRSP-style stock datasets
<code>ticker</code>	Ticker symbol of the equity (e.g., AAPL, GHC, WRO)
<code>connam</code>	Company name (long-form issuer name)
<code>secid</code>	Security ID used by OptionMetrics (option-side identifier)
<code>sdate</code>	Start date of the name–ticker association
<code>edate</code>	End date of the name–ticker association
<code>score</code>	Match quality indicator (typically 1.0 for exact match)

Knowledge Review: Straddles, Delta, and Delta-Neutral Strategies

What is a Straddle?

A *straddle* is an options trading strategy that involves simultaneously purchasing a call option and a put option on the same underlying asset, with the same strike price and expiration date. The position profits when the underlying asset exhibits significant price movement in either direction. It is commonly used as a **volatility-based strategy** that is neutral to market direction.

- A long straddle consists of buying both the call and the put.
- The payoff is maximized when the underlying asset moves far away from the strike price, regardless of direction.
- This makes the straddle ideal for traders expecting high volatility but uncertain about direction.

What is Delta?

The *delta* of an option, denoted by Δ , measures the sensitivity of the option's price to changes in the price of the underlying asset. Mathematically:

$$\Delta = \frac{\partial V}{\partial S},$$

where V is the value of the option, and S is the price of the underlying asset.

- For a **call option**, delta ranges between 0 and 1.
- For a **put option**, delta ranges between -1 and 0.
- At-the-money options generally have call delta around 0.5 and put delta around -0.5 .
- Deep in-the-money options have delta near 1 (for calls) or -1 (for puts).

Delta is also interpreted as the approximate probability that an option will expire in-the-money.

What is a Delta-Neutral Straddle?

A *delta-neutral straddle* is a modified version of the standard straddle that neutralizes exposure to small directional changes in the underlying asset. Instead of holding equal dollar amounts of the call and put, the trader assigns weights to the two legs such that the total portfolio delta is zero.

- Let Δ_C and Δ_P be the deltas of the call and put, respectively.
- Let w_C and $w_P = 1 - w_C$ be the weights assigned to the call and put.
- To achieve delta neutrality:

$$w_C \cdot \Delta_C + w_P \cdot \Delta_P = 0.$$

Deriving the Delta-Neutral Weights

Solving the delta neutrality condition yields the exact weight formula:

$$w_C = \frac{-\Delta_P}{\Delta_C - \Delta_P},$$

$$w_P = 1 - w_C.$$

Example: If $\Delta_C = 0.5$ and $\Delta_P = -0.5$, then

$$w_C = \frac{-(-0.5)}{0.5 - (-0.5)} = \frac{0.5}{1.0} = 0.5, \quad w_P = 0.5.$$

The result is an equally weighted straddle, as expected for at-the-money options. For options with asymmetric deltas, the weights automatically adjust to ensure that the portfolio has no net exposure to small movements in the underlying asset.

Please work on your code and generate the corresponding findings using these datasets. We also ask that you share your Python code for executing the back-test across the selected underlyings.

Steps to do this backtest:

Step 1: Define Option Roots and Evaluation Dates

- Let $\mathcal{R}$ represent the set of underlying equities (e.g., AAPL, MSFT, TSLA) with available listed options.
- Let $\mathcal{T}$ represent a series of evaluation dates, each corresponding to the third Friday of a calendar month (standard monthly option expiration).
- For each $(r, t) \in \mathcal{R} \times \mathcal{T}$:
 - Construct a delta-neutral straddle using one call and one put on ticker r ,
 - Ensure both options expire on the third Friday of the following month ($t' = t + 1$),
 - Record these for future return tracking.

Step 2: Extract and Filter Option Candidates

1. Retrieve all call and put option contracts for ticker r with:
 - Trading date = t
 - Expiration date = the third Friday of the next month
2. Denote the sets of calls and puts as $\mathcal{C}_{r,t}$ and $\mathcal{P}_{r,t}$, respectively.
3. Filter $\mathcal{C}_{r,t}$ to include only calls where the Black-Scholes delta falls in the range:

$$0.25 \leq \Delta_{\text{call}} \leq 0.75$$

4. From the filtered set, choose the call with:
 - The narrowest bid-ask spread,
 - A strictly positive open interest.
5. Identify the matching put in $\mathcal{P}_{r,t}$ with:
 - The same strike price and expiration date as the selected call,
 - Positive open interest.

Step 3: Construct the Delta-Neutral Straddle

- Denote the deltas of the selected call and put as Δ_C and Δ_P .
- Calculate position weights that neutralize delta exposure:

$$w_C = \frac{-\Delta_P}{\Delta_C - \Delta_P}, \quad w_P = 1 - w_C$$

- Use mid-prices to estimate execution cost:

$$C = \frac{\text{bid}_C + \text{ask}_C}{2}, \quad P = \frac{\text{bid}_P + \text{ask}_P}{2}$$

- Compute the initial cost of the straddle:

$$\text{Entry}_{\text{Straddle}} = w_C \cdot C + w_P \cdot P$$

- Save key values for future reference:

$$(r, t, K, w_C, w_P, \text{Entry}_{\text{Straddle}}, t')$$

Detailed Explanation for Step 1-3

We construct a delta-neutral straddle portfolio using matched call and put options. The following steps detail the full process, from pair selection to portfolio weighting.

(1) Select a Call and Put per Stock

On each expiration day:

- Identify options expiring in the following month.
- Keep only options with positive open interest (liquidity filter).
- Select the call option whose delta is closest to 0.5 (near-the-money).
- Find the matching put option with the same strike and expiration.
- Discard the pair if the call delta lies outside the interval $[0.25, 0.75]$.

(2) Define Variables for Weighting

Let the following variables denote:

$$\begin{aligned} \Delta_C &= \text{Call delta} \quad (\text{typically } 0.25 < \Delta_C < 0.75), \\ \Delta_P &= \text{Put delta} \quad (\text{typically negative, and } \Delta_P = \Delta_C - 1), \\ C &= \text{Midquote price of the call} = \frac{\text{bid}_C + \text{ask}_C}{2}, \\ P &= \text{Midquote price of the put} = \frac{\text{bid}_P + \text{ask}_P}{2}. \end{aligned}$$

(3) Compute Delta-Neutral Weights

We seek weights w_C and $w_P = 1 - w_C$ such that the portfolio's total delta is zero:

$$w_C \cdot \Delta_C + w_P \cdot \Delta_P = 0.$$

Derivation

Substitute $w_P = 1 - w_C$:

$$\begin{aligned}
 w_C \cdot \Delta_C + (1 - w_C) \cdot \Delta_P &= 0 \\
 w_C \cdot \Delta_C + \Delta_P - w_C \cdot \Delta_P &= 0 \\
 w_C(\Delta_C - \Delta_P) + \Delta_P &= 0 \\
 w_C(\Delta_C - \Delta_P) &= -\Delta_P \\
 w_C &= \frac{-\Delta_P}{\Delta_C - \Delta_P}, \quad w_P = 1 - w_C.
 \end{aligned}$$

Step 4: Compute Straddle Return at Expiration

- At expiration date t' , retrieve the underlying stock price $S_{t'}$.
- Compute intrinsic value of each leg:

$$\text{Payoff}_C = \max(S_{t'} - K, 0), \quad \text{Payoff}_P = \max(K - S_{t'}, 0)$$

- Calculate return:

$$R_{r,t} = \frac{w_C \cdot \text{Payoff}_C + w_P \cdot \text{Payoff}_P - \text{Entry}_{\text{Straddle}}}{\text{Entry}_{\text{Straddle}}}$$

Step 5: Construct Historical Momentum Signal

Objective: Construct a momentum signal for each underlying ticker based on its past straddle returns. This signal is used to rank assets and form momentum-sorted portfolios.

Motivation: In cross-sectional momentum strategies, we rank securities by their past performance to identify future winners and losers. Here, instead of stock returns, we use *option straddle returns*, which capture volatility-based profitability from delta-neutral positions.

1. **Collect Historical Straddle Returns:** For each ticker r , you should already have computed monthly straddle returns $R_{r,t}$ at each evaluation date t using Step 4.
2. **Define Lookback Window:** For any given evaluation month t , define a lookback window of the past 12 months:

$$\{R_{r,t-12}, R_{r,t-11}, \dots, R_{r,t-2}\}$$

3. **Exclude the Most Recent Month ($t-1$):** We intentionally exclude $R_{r,t-1}$ to avoid **short-term reversal bias**. Empirical finance research finds that performance in the most recent month often exhibits mean-reversion rather than continuation. Hence, we drop the immediate past return.
4. **Compute the Momentum Signal:** Calculate the average return over the 11-month lookback period:

$$MOM_{r,t} = \frac{1}{11} \sum_{k=2}^{12} R_{r,t-k}$$

where $R_{r,t-k}$ is the return of the straddle portfolio for ticker r evaluated k months before t .

5. **Handle Missing Data:** If fewer than 8 valid monthly returns exist in the lookback window (e.g., due to listing delay), set $MOM_{r,t} = \text{NaN}$ and exclude the ticker from the sorting step. This ensures robust signals based on a reasonable return history.
6. **Store the Momentum Values:** Store $MOM_{r,t}$ for each ticker and evaluation month as a column in your master DataFrame. This will be used for sorting tickers into quantile portfolios in Step 6.

Important Notes:

- The momentum signal is re-computed every evaluation month t and used to form portfolios at $t+1$.
- You may choose to normalize the signal cross-sectionally (e.g., z-scoring) to analyze its predictive power, but raw average returns are sufficient for ranking-based sorting.
- This formulation is inspired by the standard *12-1 stock momentum strategy*, but adapted to straddle returns in the options space.

Deliverable Requirements:

- Save a table or DataFrame showing tickers, evaluation date t , and $MOM_{r,t}$ values.
- Plot a heatmap or time-series line chart showing how $MOM_{r,t}$ evolves across time and tickers.
- Include summary statistics (mean, std, min, max) of the momentum signal across time and tickers.

Extension: Sort by IV Skew and Control for IV Effects

Objective: Assess whether the option momentum effect is driven by structural differences in implied volatility across moneyness, or whether it persists independently of the volatility curve. You will construct implied volatility skew measures at the stock-month level and test their explanatory power in relation to the momentum signal.

Required Data:

- **Option File** (daily resolution): Must include implied volatility, delta, cp_flag, strike price, expiration date, bid/ask quotes, and open interest.
- Use only options expiring on the **third Friday of the next calendar month** (i.e., 1-month-to-expiration horizon).
- Include only contracts with strictly positive open interest and valid implied volatility.

Instructions:**1. Construct IV Skew per stock-month:**

For each stock r on evaluation date t , identify:

- A **put option** with delta in the range $\Delta_{\text{put}} \in [-0.3, -0.2]$ (OTM put), and denote its implied volatility as $IV_{\Delta=0.25}$.
- A **call option** with delta in the range $\Delta_{\text{call}} \in [0.2, 0.3]$ (OTM call), and denote its implied volatility as $IV_{\Delta=0.75}$.

From each delta bucket, select the contract with the **lowest bid-ask spread** and **positive open interest**.

Then compute:

$$IVSkew_{r,t} = IV_{\Delta=0.25} - IV_{\Delta=0.75}$$

This value captures the slope of the implied volatility curve across moneyness. Larger skew values imply greater downside risk premiums embedded in option prices.

2. Control for IV Skew in Momentum Strategy:

Once $IVSkew_{r,t}$ is constructed, you must assess how it relates to momentum-based return predictability:

- **Sorting Approach:** Sort stocks into terciles or quintiles based on $IVSkew_{r,t}$. Within each group, compute long-short momentum portfolio returns and associated performance metrics (mean return, Sharpe ratio, t-statistic).
- **Regression Approach:** Estimate a panel regression each month to test the explanatory power of IV skew:

$$R_{r,t+1} = \alpha + \beta_1 \cdot MOM_{r,t} + \beta_2 \cdot IVSkew_{r,t} + \varepsilon_{r,t+1}$$

where $R_{r,t+1}$ is the straddle return in the following month, and $MOM_{r,t}$ is the 11-month average return momentum signal (skipping $t - 1$).

Interpretation:

- A significant β_2 indicates that IV skew carries predictive power.
- If β_1 remains significant after controlling for skew, it supports the robustness of the momentum effect.

3. Enhancement — Normalize by IV Level:

As a complementary test, normalize straddle returns by average implied volatility level:

$$\text{IVLevel}_{r,t} = \frac{\text{IV}_{\Delta=0.25} + \text{IV}_{\Delta=0.75}}{2}$$

$$\text{IV-adjusted Return}_{r,t} = \frac{R_{r,t}}{\text{IVLevel}_{r,t}}$$

This standardization removes biases driven by absolute volatility level differences across stocks or months.

4. Implementation Notes:

- All implied volatilities must be annualized and based on consistent maturity (1-month) and moneyness (target delta) definitions.
- Remove options with invalid IVs, zero open interest, or delta values outside the specified ranges.
- Maintain an audit log of all dropped observations, including reason for exclusion.

5. Deliverables:

- A cleaned panel with: `Ticker`, `Date`, `IV_25`, `IV_75`, `IVSkew`, and `IVLevel`.
- Plot or table showing strategy performance across IV skew groups.
- Regression output table (coefficients, t-stats).
- Commentary section interpreting the role of IV skew in explaining momentum returns.

Step 6: Form Momentum Portfolios

Objective: Use the momentum signal $MOM_{r,t}$ constructed in Step 5 to sort tickers into performance-ranked groups (quantiles), and calculate group-wise returns at time $t + 1$. The long-short portfolio formed from these groups is the core of our trading strategy.

Motivation: By ranking tickers based on past performance, we aim to exploit cross-sectional return persistence. The assumption is that straddles with higher historical returns (high momentum) will outperform those with lower historical returns (low momentum).

Step-by-Step Instructions:

1. **Choose the Number of Quantile Buckets (Q):** Typically, choose $Q = 5$ (quintiles) or $Q = 10$ (deciles). For smaller samples, $Q = 3$ (terciles) may suffice.
2. **Rank Tickers by Momentum Signal:** At each evaluation month t , take all tickers with a valid $MOM_{r,t}$ signal and sort them in descending order. The highest values are considered "winners" and the lowest are "losers".
3. **Assign Quantile Membership:** Divide the ranked tickers into Q equal-sized groups:
 - Q_H : top momentum group (e.g., top 20% for quintiles),
 - Q_L : bottom momentum group (e.g., bottom 20%).

If the number of tickers is not divisible by Q , round as needed while keeping group sizes approximately equal.

4. **Track Group Returns at Month $t + 1$:** For each group Q , compute the group-average straddle return in month $t + 1$ using:

$$R_{Q,t+1} = \frac{1}{|Q|} \sum_{r \in Q} R_{r,t+1}$$

where $|Q|$ is the number of tickers in the quantile group and $R_{r,t+1}$ is the return of the delta-neutral straddle for ticker r in month $t + 1$.

5. **Construct the Long-Short Portfolio Return:** Define the long-short return as:

$$R_{LS,t+1} = R_{Q_H,t+1} - R_{Q_L,t+1}$$

This return represents a hypothetical trade: long the highest momentum straddles and short the lowest momentum ones, held for one month.

6. **Repeat Monthly:** Repeat this process for each evaluation month t to generate a full time series of monthly long-short returns $\{R_{LS,t+1}\}$.

Implementation Notes:

- Ensure that only tickers with valid return $R_{r,t+1}$ and non-null $MOM_{r,t}$ are included in the sorting.
- Use pandas' `qcut()` or a custom quantile ranking function to assign tickers to quantile groups.
- Store Q_H , Q_L , and group-wise returns in a structured table or dictionary for analysis in later steps.

Deliverable Requirements:

- Provide a table showing, for each month t , the tickers in Q_H and Q_L , and their respective returns in $t + 1$.
- Plot the cumulative return of the long-short strategy across time.
- Save the full time series $\{R_{LS,t+1}\}$ for use in Step 7 (Performance Evaluation).

Step 7: Evaluate Strategy Performance

Objective: Quantitatively assess the performance of the delta-neutral long-short momentum strategy constructed in Step 6. The evaluation includes key statistical metrics such as average return, volatility, Sharpe ratio, and the t-statistic for significance testing.

Motivation: Backtesting is incomplete without evaluating whether the strategy delivers consistent risk-adjusted returns. This step mimics real-world investment reporting by quantifying both the magnitude and reliability of strategy performance.

Step-by-Step Instructions:

1. **Collect Long-Short Return Series:** From Step 6, you should have a time series of monthly long-short returns:

$$\{R_{LS,1}, R_{LS,2}, \dots, R_{LS,T}\}$$

where T is the number of evaluation months (e.g., number of months from 2018–2023 where returns were computed).

2. **Compute Average Monthly Return:** Calculate the mean return of the strategy over time:

$$\bar{R}_{LS} = \frac{1}{T} \sum_{t=1}^T R_{LS,t}$$

This value represents the average profit (or loss) per month for the long-short momentum trade.

3. **Compute Return Volatility:** Measure the standard deviation of the monthly returns, which captures the variability (risk) of the strategy:

$$\sigma_{LS} = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (R_{LS,t} - \bar{R}_{LS})^2}$$

A higher σ_{LS} implies greater risk and more return variability across time.

4. **Compute the Sharpe Ratio:** The Sharpe ratio is the industry-standard metric for evaluating risk-adjusted performance:

$$\text{Sharpe Ratio} = \frac{\bar{R}_{LS}}{\sigma_{LS}}$$

This tells us how many units of return the strategy earns per unit of risk. A Sharpe ratio above 1 is generally considered acceptable; above 2 is considered strong.

5. **Compute the t-Statistic:** The t-statistic helps us test whether the average return is statistically different from zero:

$$t\text{-statistic} = \frac{\bar{R}_{LS}}{\sigma_{LS}/\sqrt{T}}$$

A rule of thumb in asset pricing is that $|t| > 2$ implies statistical significance at the 5% level (under standard assumptions of i.i.d. returns).

6. **(Optional) Annualize Metrics:** If desired, convert monthly metrics to annual units for comparability:

$$\begin{aligned}\bar{R}_{LS}^{\text{annual}} &= 12 \cdot \bar{R}_{LS} \\ \sigma_{LS}^{\text{annual}} &= \sqrt{12} \cdot \sigma_{LS} \\ \text{Sharpe}^{\text{annual}} &= \frac{\bar{R}_{LS}^{\text{annual}}}{\sigma_{LS}^{\text{annual}}} = \text{Sharpe Ratio}\end{aligned}$$

Deliverable Requirements:

- Report the following in a performance summary table:
 - Average monthly return ($\bar{R}_{LS}$)
 - Monthly volatility (σ_{LS})
 - Monthly Sharpe ratio
 - t-statistic
 - (Optional) Annualized versions of the above metrics
- Plot the cumulative performance of the strategy:

$$\text{Cumulative Return}_t = \prod_{\tau=1}^t (1 + R_{LS,\tau}) - 1$$

- Provide a histogram of $R_{LS,t}$ to show return distribution.
- Discuss whether the performance is economically and statistically meaningful.

Extension: VIX Integration (Market Volatility Control)

Objective: Evaluate whether the strategy's performance varies under different levels of market-wide volatility, as proxied by the CBOE VIX index. The goal is to determine whether our momentum-based option strategy is conditional on macro volatility states, and whether portfolio performance is more favorable during periods of heightened investor uncertainty.

Instructions:

1. **Data Source:** Obtain historical daily closing levels of the CBOE VIX Index (Dropbox Folder).
2. **Monthly Aggregation:** For each calendar month t , compute the VIX level as the average of daily VIX values during that month. Denote this monthly average as VIX_t .
3. **Partitioning the Sample:** Split the full set of evaluation months into two distinct volatility regimes:
 - **High Volatility Regime:** All months where $VIX_t > \text{median}(VIX)$ over the full sample.
 - **Low Volatility Regime:** All months where $VIX_t \leq \text{median}(VIX)$.
4. **Conditional Back-Test:** Recompute your strategy's long-short returns and performance statistics within each subsample:
 - For each regime, calculate the average long-short return $\bar{R}_{LS}^{\text{regime}}$, standard deviation $\sigma_{LS}^{\text{regime}}$, Sharpe ratio, and t-statistic.
5. **Visualization:** Create a comparative chart or bar plot that clearly displays performance metrics under both high and low VIX regimes.

6. **Optional - Regression Control:** Incorporate the VIX level or change in VIX as a control variable in a predictive regression of long-short returns:

$$R_{LS,t+1} = \alpha + \beta \cdot \text{VIX}_t + \epsilon_{t+1}$$

where $R_{LS,t+1}$ is the return on the long-short momentum portfolio in month $t + 1$, and VIX_t is the average VIX level observed in month t .

7. **Interpretation:** Provide a brief discussion on whether the strategy's effectiveness is conditional on volatility regime. Are the returns stronger when the market is fearful? Does the VIX serve as a useful risk filter for position sizing or capital allocation?

Deliverable: Your updated slide deck should include the partitioned performance statistics, relevant charts, and a brief commentary on the impact of market volatility on strategy performance. Code logic and regression results should be documented clearly in your Jupyter Notebook or Python script.

Step 8: Risk Management and Position Sizing

Objective: Enhance portfolio risk control by incorporating volatility-adjusted position sizing. This aligns with hedge fund industry practices where risk is budgeted across positions to avoid concentration in volatile names.

Motivation: Without scaling, each position contributes equally in nominal terms regardless of its risk. However, more volatile straddles may dominate portfolio returns and risk. This step ensures each position contributes more evenly to total portfolio volatility.

1. **Collect Historical Straddle Returns:** For each ticker r , compute the last 11 months of historical straddle returns:

$$\{R_{r,t-2}, R_{r,t-3}, \dots, R_{r,t-12}\}$$

2. **Compute Rolling Standard Deviation:** Calculate the rolling standard deviation of straddle returns for each ticker r using a 11-month window:

$$\sigma_{r,t} = \text{std}(\{R_{r,t-k}\}_{k=2}^{12})$$

3. **Define Inverse-Volatility Weight:** Compute a scaling factor based on inverse volatility:

$$w_{r,t}^{\text{risk}} = \frac{1}{\sigma_{r,t}}$$

4. **Normalize Weights Within Each Portfolio:** For tickers selected in each group (top and bottom momentum quantiles), normalize their inverse volatility weights so that they sum to one:

$$\tilde{w}_{r,t} = \frac{w_{r,t}^{\text{risk}}}{\sum_{r \in Q} w_{r,t}^{\text{risk}}} \quad \text{for } Q \in \{Q_H, Q_L\}$$

5. **Apply Weights to Returns:** When computing group returns in Step 6, use these normalized weights instead of equal-weighting:

$$R_{Q,t+1} = \sum_{r \in Q} \tilde{w}_{r,t} \cdot R_{r,t+1}$$

6. **Continue with Long-Short Return Calculation:** Compute the long-short return as usual:

$$R_{t+1}^{\text{LS}} = R_{Q_H,t+1} - R_{Q_L,t+1}$$

Deliverable Requirements:

- Include side-by-side performance results comparing equal-weighted and volatility-adjusted portfolios.
- Report Sharpe ratio and t-statistics for both methods.
- Discuss any differences in return stability, drawdowns, and concentration risk.

Step 9: Transaction Cost and Slippage Adjustment

Objective: Incorporate realistic assumptions about trading frictions into the backtest, including transaction costs and slippage, to better reflect real-world execution. This step is crucial for evaluating the true economic value of the strategy.

Motivation: Option strategies often incur significant costs due to wide bid-ask spreads and low liquidity. Ignoring these frictions can overstate returns and Sharpe ratios. A hedge fund-level backtest must account for these hidden costs to assess robustness.

1. **Define Transaction Cost Parameters:** Use the following assumptions:

- Slippage: Assume execution at the *midquote price* for ideal conditions, and penalize by a fixed slippage rate for more realistic scenarios.
- Round-trip cost: Set to 5 basis points (bps) of notional for each leg as a baseline.
- Bid-ask spread: Estimated implicitly by the difference between `best_offer` and `best_bid`.

2. **Compute Execution Prices with Slippage:** Adjust the midquote prices of the call and put used in entry and exit as follows:

$$\text{MidPrice}_{\text{entry}} = \frac{\text{bid}_{\text{entry}} + \text{ask}_{\text{entry}}}{2}$$

$$\text{EffectivePrice}_{\text{entry}} = \text{MidPrice}_{\text{entry}} \times (1 + \text{slippage_rate})$$

where `slippage_rate` is typically 0.001 (0.1%).

Repeat this for the exit price at expiration:

$$\text{EffectivePayoff} = \text{GrossPayoff} \times (1 - \text{exit_cost_rate})$$

3. **Apply Round-Trip Cost Deduction:** Deduct the assumed round-trip cost from the return calculation:

$$\text{RoundTripCost} = \text{EntryStraddle} \times \text{cost_rate}$$

where `cost_rate` = 0.001 per leg (0.2% total for two legs).

4. **Adjust Return Formula:** Incorporate costs into return:

$$R_{r,t}^{\text{net}} = \frac{w_C \cdot \text{Payoff}_C + w_P \cdot \text{Payoff}_P - \text{EntryStraddle} - \text{RoundTripCost}}{\text{EntryStraddle}}$$

5. **Sensitivity Analysis (Optional):** Run sensitivity checks using different values of slippage and cost rate (e.g., 0.05%, 0.10%, 0.25%).

Deliverable Requirements:

- Compare strategy returns with and without cost adjustments.
- Plot monthly return distributions before and after frictions.
- Report impact on Sharpe ratio, drawdowns, and net alpha.
- Comment on whether the strategy still performs after accounting for realistic transaction costs.

Weekly Progress Deliverables

Interns are expected to submit the following three deliverables upon completing the Option Momentum Task:

1. **Presentation Slide Deck (PPT):**

A well-structured PowerPoint presentation summarizing your methodology, portfolio construction logic, key findings, performance metrics (Sharpe, t-stat), and any visual insights (return distribution, portfolio P&L, etc.). This should be formatted for internal review.

2. **Python Code File (.py):**

A standalone Python script or Jupyter Notebook file that contains your full code for data loading, processing, back-testing, and performance evaluation. The code should be clean, modular, and commented to ensure readability and reproducibility.

3. Code Output Report (PDF):

A compiled PDF report that contains both the Python code and its executed outputs. This should be exported from a Jupyter Notebook or similar environment and include:

- All major code blocks (with headings)
- Printed results and tables
- Plots and performance charts

This document should allow the reviewer to understand both the logic and the outcomes without needing to run the code manually.